

Cloud Based Fog-Edge Data Center Thermal Monitoring System

Abhishek Bheemaraddi Mudaraddi

Student ID : 24221163

Fog and Edge Computing ,MSc in Cloud Computing

National College of Ireland Dublin, IRELAND

Email : x24221163@student.ncirl.ie

Application URL : <http://fnc-thermal-cloud.eba-hh7pmadx.us-east-1.elasticbeanstalk.com/>

GitHub Repo Link: <https://github.com/AbhishekMudaraddi/FogAndEdge>

Abstract – The report introduces a cloud-based thermal monitoring application that is deployed based on the principles of Fog and Edge computing, in AWS. The project point a solution to a real life operation problem, which can be summarized as how to convert continuous and high levels of environmental telemetry to provide a reliable and interpretable thermal risk awareness across the distributed regions of data centers. Instead of moving raw data to cloud storage and interpret it at visualization time only, the system adds a fog computation layer to add extra content and contextualize telemetry as it passes on. The solution is a mix of an edge simulator tool, an event driven fog node, and a scalable cloud backend.

In edge level, there are five sensor streams which are configured to be dispatched at a variable dispatch allowing normal and accelerated demonstration behavior. The queue events are consumed at the level of fog, transformed into enriched records of operations with derived-state computation, such as indicating overheating and to-cooling-risks semantics. Cloud level Enriched records are stored and made available via region, availability-zone, rack-oriented dashboards that can be used to support situational awareness and detailed diagnostics. The thermal escalation and recovery are enabled by a controlled simulation mechanism making the architecture auditable or presentable in case of presentations and testing.

Elastic Beanstalk, SQS, Lambda, DynamoDB, CloudWatch Dashboard, Cesium library, Chart.js and GitHub Actions CI/CD are all used in the implementation. The main point is that, Fog-Edge-Cloud pipeline provides better decoupling, resilience and explainability over cloud-only telemetry flow. The project has thus not only a technical feasibility, but also provides the architectural usefulness of fog-layer intelligence within an operational monitoring system.

Key Terms : Fog computing, Edge computing, AWS Lambda, Amazon SQS, DynamoDB, Elastic Beanstalk, thermal monitoring, CI/CD.

I. INTRODUCTION

Throughout the data center, there must be consistent thermal awareness to inhibit overheating-related breakpoints, wastefulness in cooling and deterioration of the services. Practically, not just gathering measurements but also transmitting them to the appropriate processing tier in time is challenging. A basic cloud-based telemetry model can be useful to store values, but it tends to place all interpretation to the centralized backend logic, or dashboard code, which is highly coupled and may fail to provide operational clarity when data rates peak, or abnormal conditions evolve rapidly.

A stronger architectural response is provided by fog and edge computing. Edge resources have the ability to generate telemetry near-context of sources and other nodes in the fog computing generate intermediate computation to transform raw values into useful states of operations then long-term persistence to the clouds [1], [2]. This stack architecture enhances response and enables cloud services to specialize in sustainable storage, cross-region analysis and visualization to end-users as opposed to initial semantic conversion.

The current project creates a full Fog-Edge-Cloud application to monitor temperature-related risk in several simulated areas. The system is closely focused around educational and operational requirements: fulfilling module requirements, exercising architectural separation, and facilitating realistic failure/recovery demonstrations. The system consists of an editable edge simulator, a coded fog processing node, and a cloud back-end having multi-level dashboards. It also has a simulation control mechanism which is able to force region-level critical states and can provide repeatable demonstration of full-pipeline behavior.

On a domain basis, the project shifts to observability into actionable raw telemetry. The isolated sensor values are not generally required by the operators; interpreted states along with trends and transition visibility are required by the operators. The layer of mists thus calculates derived risk flags and contextual features prior to cloud persistence that enhances the interpretability and uniformity of downstream dashboard.

Academically, the project proves that not only are conceptual architecture terms used to deploy labels, but also labels to the engineering layer of specific responsibility are deployed. The resulting system offers a tangible, traceable reference point of termination of edge generation, termination of fog intelligence, and cloud infrastructure provision of scalable persistence and visualization.

II. OBJECTIVES AND REQUIREMENTS.

The main aim is to architect and implement an entire stack cloud application illustrating the concepts of fog and edges under a real monitors scenario. The functional scope obtained should comprise 3-5 type of mock sensor generation, processing telemetry with coded logic on the fog nodes, and records dispatch to a backend web service having the capability of responsive visualization. The architecture should also be shown to be scaled by the usage of patterns like queue decoupling, event-driven compute and controlled cloud services.

The second goal is architectural explainability. Since this project is evaluated in a module environment the solution is required not just to start but also clearly demonstrate why each layer is present and what is the ownership it holds. This is why the implementation focuses on separation of concerns: edge to telemetry generation, fog to add context, and cloud to persist and to provide user monitors.

One of the third goals is demonstration preparedness. Most monitoring systems display only nominal-state charts, this project provides controlled critical-state simulation to reproduce high-temperature events. This allows the assessment of both the escalation and recovery behavior without waiting until anomalies occur randomly.

The system is developed to achieve these goals by five streams of sensors, SQS-based queueing, Lambda-based fog processing, DynamoDB persistence and Flask based cloud APIs/dashboards. CloudWatch Dashboard facilitates observability of the pipeline processes, whereas CI/CD using GitHub Actions can guarantee reproducible deployments and consistency of configuration. Consequently, the project meets the outcomes of the modules as well as offers more practical details about the behavior of fog and edge cases.

III. SYSTEM ARCHITECTURE

A. Three-Layer Architecture

The architecture implements a three-layered architecture: edge generating, fog processing and cloud serving. This stratification is the key component of the design philosophy of the project. The edge layer has the role of generating high-frequency telemetry along with the source context (region, AZ, rack, sensor type, timestamp). It is the role of the fog layer to transform those values into records that can be interpreted operated upon. The cloud layer will be in charge of offering durable storage, aggregation, and visualization.

Instead of per-reading writings into the database, the edge simulator publishes batched payloads. The semantic and control boundary where the raw signals are converted to enriched telemetry with risk semantics is the fog node, implemented on Lambda and SQS trigger mapping..

There are three design patterns which are center-stage in this architecture. First, ingestion decoupling of producers to consumers by queues, isolates the ingestion of downstream processing capacity [3]. Second, event-based fog implementation enables scalability of compute with load at low idle overhead [4]. Third, cloud aggregation that is read-optimized enables dashboards to obtain succinct, targeted summaries without scanning the raw data repeatedly, at the client end.

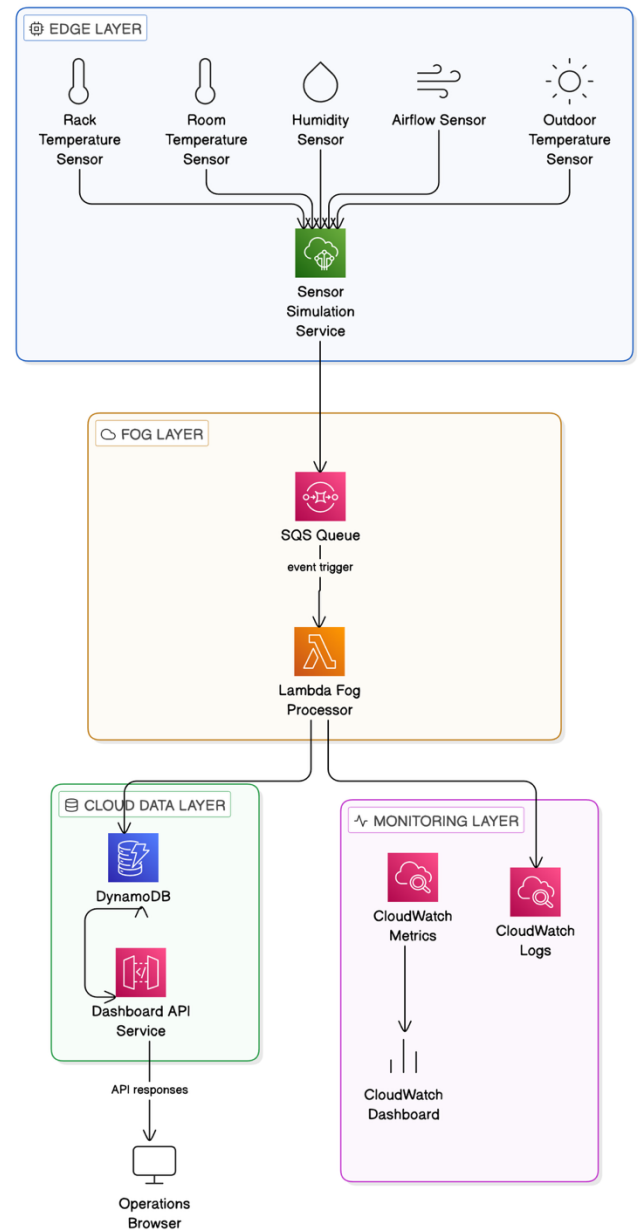


Figure 1 – The system architecture of the Application developed showing the 3 layers of Edge layer, Fog layer and the Cloud data layer and also the application involving the monitoring layer of cloud watch.

B. Service Mapping and Design Justification

Elastic Beanstalk supports edge and cloud Flask applications since it can be applied to manage application lifecycle, environment variables and have simple deployment processes [5]. In the case of this project, Elastic Beanstalk allows operational overheads to be minimized and instead focus on complexity in architecture behavior instead of provisioning infrastructure.

SQS is chosen as the transport boundary due to its ability to give long-lasting buffering and decoupled producer timing remains linked to processor execution. The sensor bursts might be related to a higher dispatch cadence, simulation mode or temporary processing delay.

Configuration of lambda is a result of selecting the event-oriented telemetry transformation that it suits best in this kind of application. The nodes in this project are virtual/coded fog nodes, and Lambda offers a controllable implementation of this theory that expands in response to the arrival rate, message-wise. This assists in embodying fog computation as a transitional, elastic service instead of a fixed and uniform service.

The choice of DynamoDB to persist telemetry is because of foreseeable low-latency access, high write throughput, and TTL-based retention [6]. Given that time-series data is continuously generated in this project, bounded retention can be used to ensure that storage utilization is also practical.

The evolution of edge simulation rules is possible with the same cloud query APIs. Rules of enriching fog are able to be tuned without any adjustments of dashboard logic. Cloud dashboards are able to add views without altering edge dispatch. This separation minimized regression risk during development and facilitated quick development of region/AZ/rack visualization and simulation controls.

C. Overall Data Flow

The total figure is to be interpreted as a stream of connections: Edge Simulator – SQS - Lambda Fog Processor - DynamoDB - Cloud API - Dashboard UI. This flow describes the business data flow. An independent but parallel operational channel delivers run-time measurements and logs to CloudWatch, and CloudWatch Dashboard delivers health visibility.

The edge stage builds organized batches of JSON comprising of sensor measurements and context location. The queue stage offers decoupling of delivery and buffering in time. The stage of the fog converts the measurement-centric into the state-centric data through generation of risk semantics. Storage stage stores enriched items to be queried. The API level develops response models, applicable in dashboards. The UI phase will translate these responses to global overview and AZ diagnostics, rack level comparison and critical states visualization.

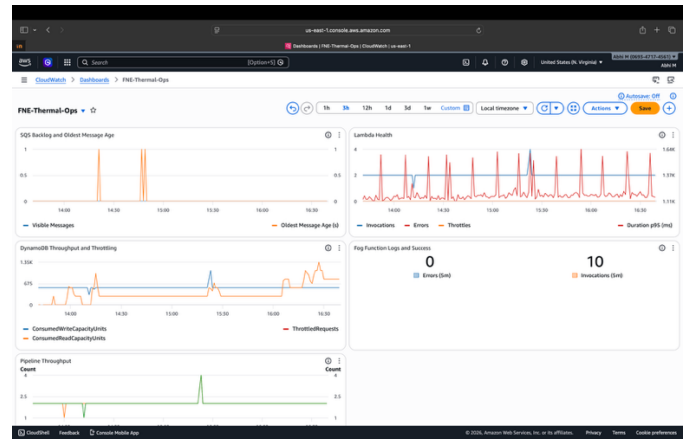


Figure 2- Dashboard of AWS CloudWatch implementation

It is deliberately a one direction telemetry ingestion and two way only at API/UI communication level. These disciplines of flow simplify the architecture, and aid in ensuring that decisions which are being monitored are based on persisted, fog-enriched records, as opposed to transient client state.

IV. IMPLEMENTATION

A . Edge Layer: Sensor Creation and Dispatch

The edge layer is a Python/Flask service with a periodic worker which produces telemetry in cycles. The readings of five proposed simulated sensors (rack temperature, room temperature, humidity, airflow, and outdoor temperature) are produced by each cycle. Rack identity, availability-zone identity and region identity are tagged to the generated data to allow hierarchical aggregation to be done later on. Such location tagging is crucial in proving the realistic operational perspectives as opposed to planar sensor diagrams.

The values, frequency and dispatch controls of the generator are adjustable. Design wise, this enables the differentiating of environment settings to classroom demonstration mode, normal monitoring mode, and stress demonstration mode, by simply altering the settings. This type of runtime configurability is significant to edge computing systems, in which it is often required to optimize behavior without re-deploying the system.

```
def _random_value(sensor_type: str, rack_id: str, active_prefixes: set[str]) -> float:
    if sensor_type == "rack_temperature":
        rack_prefix = rack_id.split("-", 1)[0].lower()
        if rack_prefix in active_prefixes:
            return round(random.uniform(SIM_CRITICAL_RACK_TEMP_MIN, SIM_CRITICAL_RACK_TEMP_MAX), 2)
        return round(random.uniform(NORMAL_RACK_TEMP_MIN, NORMAL_RACK_TEMP_MAX), 2)
    if sensor_type == "room_temperature":
        return round(random.uniform(18.0, 28.0), 2)
    if sensor_type == "humidity":
        return round(random.uniform(30.0, 60.0), 2)
    if sensor_type == "airflow":
        return round(random.uniform(1.2, 3.5), 2)
    if sensor_type == "outdoor_temperature":
        return round(random.uniform(-5.0, 35.0), 2)
    return 0.0
```

Figure 2- The part of the edge code that is responsible for generating the sensor values for simulation

Telemetry would be sent to SQS with the consolidation as opposed to databases. The choice constitutes one of the principles of core edges/fog: the focus of edge devices must be placed on sensing and dispatch reliability, whereas interpretation and storage semantics are placed in the upstream layers of processing. Queue-first dispatch also offers burst tolerance in the event of a burst in generation relative to downstream processing

The edge service has a simulation interface (‘/sim’) to initiate controlled thermal escalation based on what regions are selected. When a simulation is turned on the critical range is shifted in rack temperature values and other signals are still produced. This renders critical-state behavior predictable and enables entire pipeline response to be shown, with dashboard transitions and recovery clearing.

B. Fog Layer: Lambda Processing.

The visibility into the fog layer is introduced as a Lambda function, which links the SQS with the event source mapping. This implies that every processing cycle is activated by the arrival of queues and not polling by a clock. Architecturally, this is a good match to the theory of fog computing in general since it implements elastic near-ingesting computation into the data path without permanently executing compute instances.

Within each batch, sensor payloads are interpreted and the presence of mandatory fields is validated, as well as sensor relationships evaluated to obtain operational context. Derived indicators consist of overheating, risk of cooling-failing, and semantics of static risk indicators. Rather than driving these interpretations to the front-end code, they are calculated during the first step of the fog processing and then they are saved in first-class fields. This provides a uniform interpretation of all the dashboard views.

Prep of persistence-friendly item structures is also done by the fog node which writes them in DynamoDB through batch. All items contain metadata telling which records are retention metadata such that TTL lifecycle control can automatically expire stale telemetry. This plays a critical role operationally in continuous monitoring loads where the volume of data continually increases.

To facilitate the observability, troubleshooting, and performance checks, structured logging is produced during the process of processing the fog. Since the semantic focus of the architecture is fog, lower-level logging is particularly useful in debugging errors in signal mapping to the dashboard.

C. Cloud Backend, Front-End, and Libraries.

The cloud backend is deployed as a Flask application with REST-like endpoints to provide latest summaries, historical sensor trends, AZ rollups and rack-level diagnostics. The queries are based on persisted data that has been enhanced with fog, and as such API responses are operationally meaningful when being provided to clients. This backend aggregation

model maintains a simple client logic and minimizes interpretation rules replication.

The patterns of DynamoDB access can be configured in such a manner so that they enable the access of recent state and time series trend reads to the region/az/rack scopes in the DynamoDB. Aggregation helps calculate values like average rack temperature, hottest AZ and rick AZ counts. These metrics on the server are directly powered by dashboards and ensure the UI remains responsive as more data are added to the charts

Front-end is rendered by templates that are server-rendered and JavaScript refresh. CesiumJS is applied to the global 3D infrastructure visualization providing the ability to mark the regions and significant changes of visual states on a globe interface. the visualization of trends and comparisons, such as rack-level comparison and region-by-region side-by-side comparison, are done using Chart.js.

The user experience has been divided to include Home, AZ Overview, Compare and region diagnostics. Home perspective focuses on situational awareness by globally interacting and giving crucial notifications. The AZ and regional diagnostics views embrace analytical drill-down. Notably, these views are all inspired by cloud recordings generated by mood enrichment, guaranteeing a single source of fact between backend and UI.

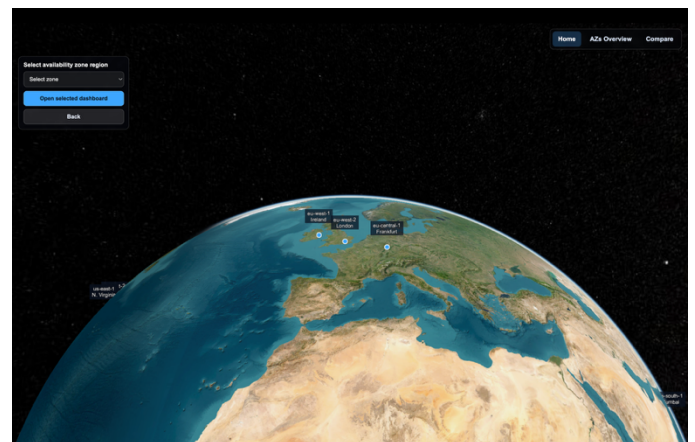


Figure 3 – A globe visulation that is created using the available library called Cesium

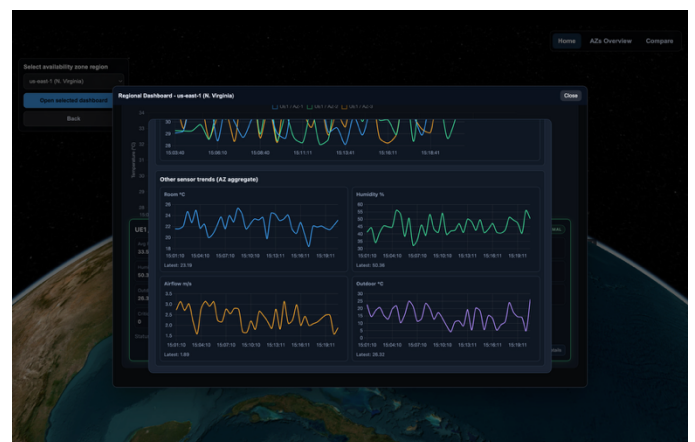


Figure 4- The Dashboard showing all the sensors simulated data in the form of graphs visual.

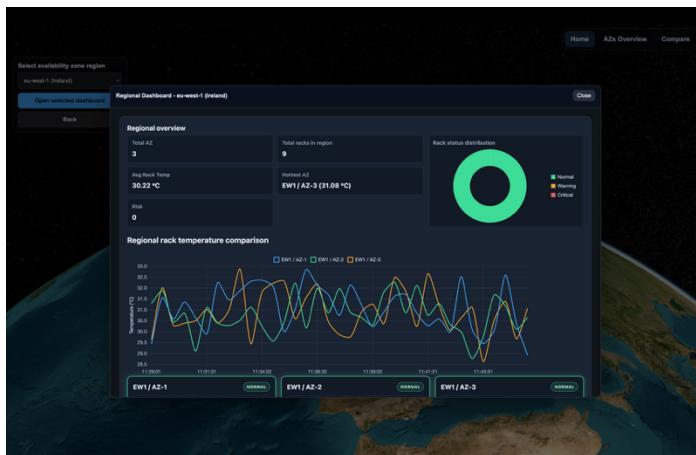


Figure 4 – A visual UI for representing the sensors data.

D. Continuous Integration and Deployment

CI/CD is enforced by GitHub Actions and designed to deploy edge and cloud apps standalone but can optionally augment the pipeline family with fog functions. This reflects the separation of concerns in the architecture at deployment level and allows iterating on the layers of the architecture without redeploying the entire system.

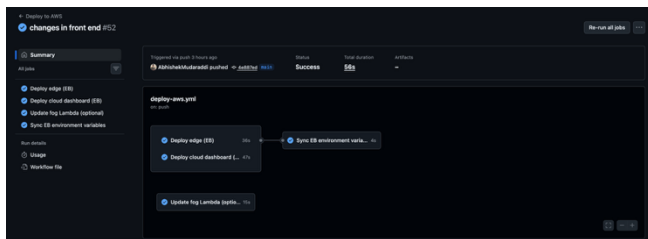


Figure 5 – GitHub action CICD pipeline responsible for Deploying the application and Lambda function.

The workflow is used to package, deploy as Elastic Beanstalk environments and coordinate significant environment variables (queue URL, table identifiers, cadence controls, and rack/region scope values). The project minimizes the number of deployment errors by documentation as well as enhancing consistency in configuration among the environments.

CI/CD was essential in a project lifecycle viewpoint in fast refinement. There was a large number of redeployments to improve dashboard behavior, simulation controls and region mapping. These updates were repeatable, and auditable due to automated pipelines.

GitHub Repository Link:

<https://github.com/AbhishekMudaraddi/FogAndEdge>

VI. CONCLUSIONS AND FINDINGS,

This initiative shows that fog and edge architecture have overt practical advantages over a divinely cloud-only telemetry path towards thermal monitoring set-ups. Decoupling using queues enhances resilience of ingestion, enriching systems state with a fog layer improves interpretability, and giving operational awareness through cloud layer aggregation and visualization is

scalable. The architecture thus not only acts as a data pipeline, but as a structured decision-support pipeline.

Among the key discoveries is that the location of interpretation is just as important as data storage. Semantics computing in fog stage resulted in more reliable dashboards and less client-side ambiguity. The second observation is that explicit region/AZ/rack hierarchy in data modelling can greatly enhance usability as a user is able to navigate between global context and detailed diagnosis without losing track.

The third insight is that, when it comes to robust evaluation, controlled simulation is necessary. The capability to induce region specific critical temperature behavior facilitated reproducible tests of escalation, sustained critical state and recovery. In the absence of such a capability, the Architecture validation would be based on a random behavior of values and would be much less compelling.

Reflectively, the project solidified that the concept of fog and edges are better understood when they are actualized as hard limitation of responsibility, instead of fuzzy intermediate terms. Separate edge, fog services, and cloud services development revealed actual integration problems, in particular, coordination of configurations and uniform metadata. The resolution of those problems enhanced the knowledge on the behavior of a distributed system than just isolated coding exercises.

The other resemblance is that CI/CD is not merely a convenience of delivery but enabler of architectural. Repeat iterative changes at cross-layered levels were feasible since workflows of deploying were automated and repeatable. This saved the time of debug, and the end system was more stable to demonstrate.

In general, the project manages to make the module theory into an applied system, which is technically sound, stress-testable, and can be extended upon in the future.

Future Work

The current enhancements will be the future predictive risk rating based on trend models to ensure that potential thermal accidents are surfaced prior to threshold violation. The other direction is a better-policy-driven logic in the fog layer, in which region-specific cooling semantics may be dynamically used. The third way is enhanced long-window analytics with optimal historical querying and feature extraction to detect anomalies.

Other engineering enhancements involve increased automated checks of edge generation quality, robustness of the fog-enrichment, and API/dashboard-equivalence. Stricter IAM segmentation, deployment gates with audit focus and environment drift identifying within the CI/CD pipelines is also used to enhance security and governance.

Lastly, the architecture may be scaled to hybrid deployment in which some cloud aggregation and global dashboarding is maintained by having the chosen fog logic that is deployed closer to physical locations. It would also enhance the principle

of edge-to-fog locality and give a natural extension of this project to higher degree of fog computing applications.

VII . REFERENCES

- [1] [F. Bonomi, R. Milito, J. Zhu, and S. Addepalli, “Fog Computing and Its Role in the Internet of Things,” in *Proceedings of the First Edition of the MCC Workshop on Mobile Cloud Computing*, 2012, pp. 13–16.](#)
- [2] [M. Aazam, S. Zeadally, and K. A. Harras, “Fog Computing Architecture, Evaluation, and Future Research Directions,” *IEEE Communications Magazine*, vol. 56, no. 5, pp. 46–52, 2018.](#)
- [3] [Amazon Web Services, “Amazon SQS Developer Guide,” AWS Documentation. \[Online\]. Available: <https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/welcome.html>](#)
- [4] [Amazon Web Services, “Using AWS Lambda with Amazon SQS,” AWS Documentation. \[Online\]. Available: <https://docs.aws.amazon.com/lambda/latest/dg/with-sqs.html>](#)
- [5] [Amazon Web Services, “AWS Elastic Beanstalk Developer Guide,” AWS Documentation. \[Online\]. Available: <https://docs.aws.amazon.com/elasticbeanstalk/latest/dg/Welcome.html>](#)
- [6] [Amazon Web Services, “Using DynamoDB Time to Live \(TTL\),” AWS Documentation. \[Online\]. Available: <https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/TTL.html>](#)
- [7] [Chart.js Contributors, “Chart.js Documentation.” \[Online\]. Available: <https://www.chartjs.org/>](#)
- [8] [CesiumGS, “CesiumJS Documentation.” \[Online\]. Available: <https://cesium.com/platform/cesiumjs/>](#)
- [9] [GitHub Docs, “GitHub Actions Documentation.” \[Online\]. Available: <https://docs.github.com/actions>](#)

